ASSIGNMENT - 10
COMPUTER NETWORKS
ID - 12341550

PART A : DNS-Assisted UDP Video Streaming System using Socket Programming

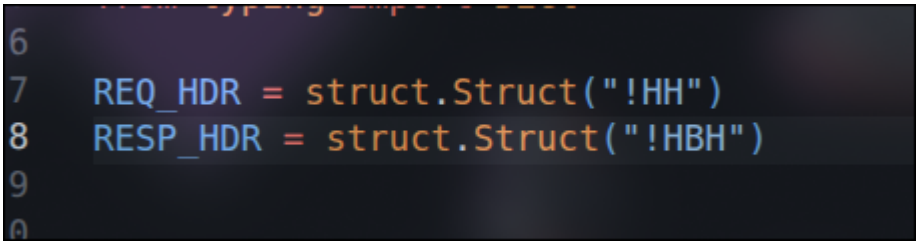
Language : Python

DNS Resolution (TCP):
The DNS Server (dns_server.py) operates over TCP. It listens for incoming queries, unpacking a 2-byte sequence representing the length of the domain name, followed by the domain string itself. It references a local **dns_records.txt** file to resolve the domain to an IP address and Port. The response is packed into a strict binary format containing a 2-byte transaction ID, 2-byte IP address length, the string IP address, and a 2-byte port number (**!HBH format**).

Video Streaming (UDP):
The Video Server (video_server.py) uses **OpenCV** (cv2) to capture webcam frames. To adhere to standard MTU limits and prevent IP fragmentation, each JPEG-encoded frame is chunked into fragments of **1200** bytes. Each UDP datagram encapsulates a custom header (!**IHHH format**):

->Total frame length (4 bytes)
->Current chunk offset (2 bytes)
->Payload length (2 bytes)
->End-of-frame flag (2 bytes)

Keep-Alive Mechanism:
To ensure the server only streams while the client is active, a multi-threaded approach is used on both ends. The client continuously sends a **KEEPALIVE** message over **UDP** to the server. The server tracks the timestamp of the last received message. If the server does not receive a keep-alive within a **200ms** threshold, it safely suspends transmission, resuming seamlessly when the client reconnects.



DNS message format definitions - dns_server.py

```python
while True:
    conn, addr = srv.accept()
    with conn:
        try:
            hdr = recv_exact(conn, REQ_HDR.size)
            txid, qname_len = REQ_HDR.unpack(hdr)
            if qname_len == 0 or qname_len > 255:
                continue

            qname_raw = recv_exact(conn, qname_len)
            qname = qname_raw.decode("utf-8", errors="replace")
            ip = records.get(qname)
            if ip is None:
                resp = RESP_HDR.pack(txid, 1, 0)
                send_all(conn, resp)
                print(f"MISS txid={txid} domain={qname} from {addr[0]}:{addr[1]}")
            else:
                ip_b = ip.encode("utf-8")
                resp = RESP_HDR.pack(txid, 0, len(ip_b)) + ip_b
                send_all(conn, resp)
                print(f"HIT txid={txid} domain={qname} -> {ip}")
        except Exception as exc:
            print(f"request error from {addr}: {exc}")
```

DNS request parse and response build path - dns_server.py

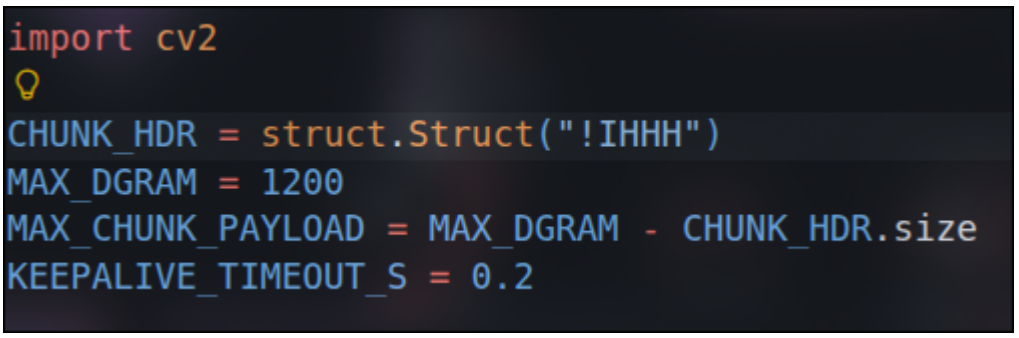
```python
def resolve_domain(dns_ip: str, dns_port: int, domain: str, txid: int) -> str:
    qname = domain.encode("utf-8")
    req = REQ_HDR.pack(txid, len(qname)) + qname

    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.connect((dns_ip, dns_port))
        s.sendall(req)

        resp_hdr = recv_exact(s, RESP_HDR.size)
        r_txid, status, ip_len = RESP_HDR.unpack(resp_hdr)
        if r_txid != txid or status != 0 or ip_len == 0:
            raise RuntimeError("domain resolution failed")

        ip_raw = recv_exact(s, ip_len)
        return ip_raw.decode("utf-8")
```

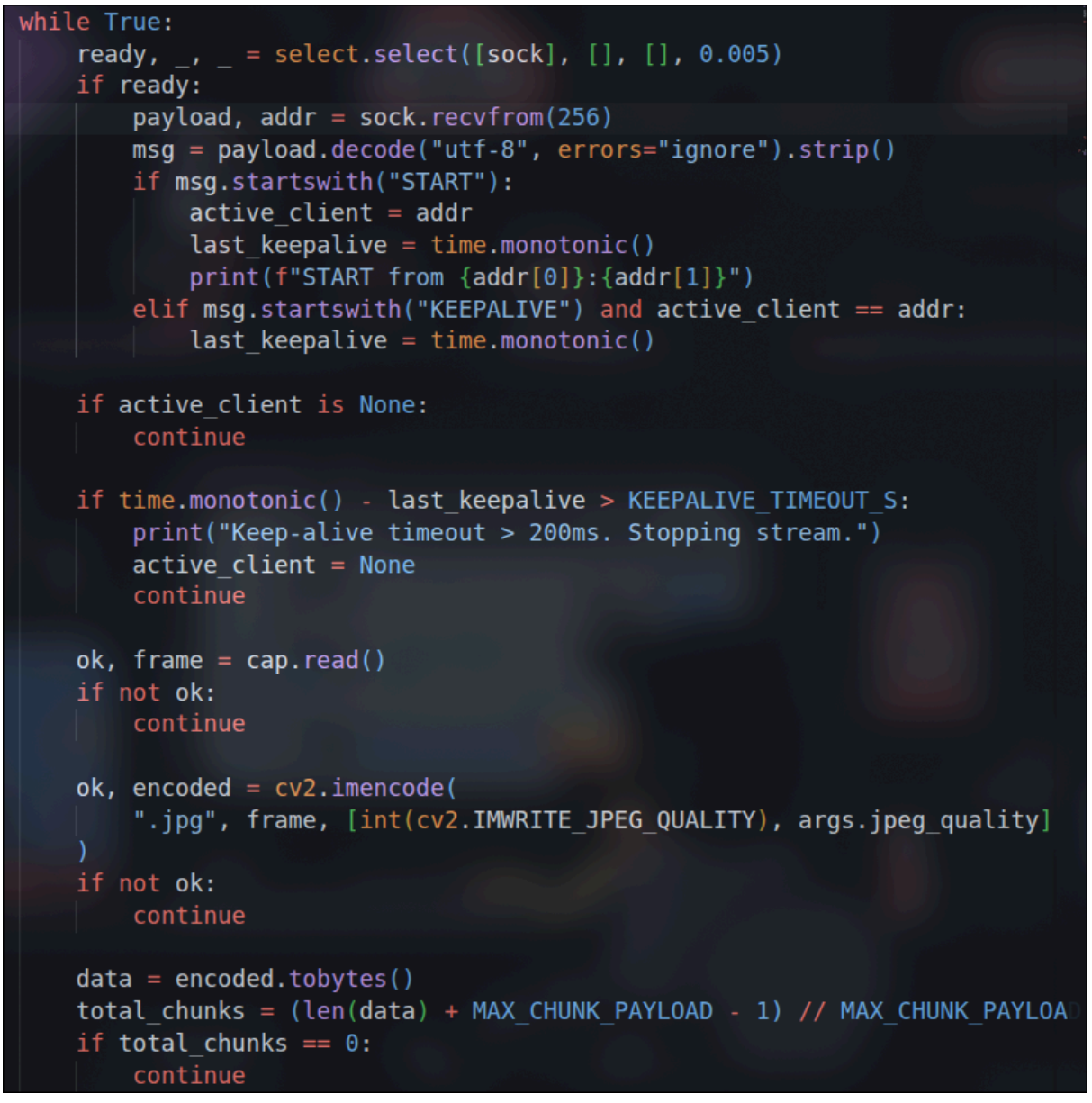
Client DNS resolve transaction - client.py



```
import cv2

CHUNK_HDR = struct.Struct("!IHHH")
MAX_DGRAM = 1200
MAX_CHUNK_PAYLOAD = MAX_DGRAM - CHUNK_HDR.size
KEEPALIVE_TIMEOUT_S = 0.2
```

UDP chunk header constants and keep-alive timeout - video_server.py



```
while True:
    ready, _, _ = select.select([sock], [], [], 0.005)
    if ready:
        payload, addr = sock.recvfrom(256)
        msg = payload.decode("utf-8", errors="ignore").strip()
        if msg.startswith("START"):
            active_client = addr
            last_keepalive = time.monotonic()
            print(f"START from {addr[0]}:{addr[1]}")
        elif msg.startswith("KEEPALIVE") and active_client == addr:
            last_keepalive = time.monotonic()

    if active_client is None:
        continue

    if time.monotonic() - last_keepalive > KEEPALIVE_TIMEOUT_S:
        print("Keep-alive timeout > 200ms. Stopping stream.")
        active_client = None
        continue

    ok, frame = cap.read()
    if not ok:
        continue

    ok, encoded = cv2.imencode(
        ".jpg", frame, [int(cv2.IMWRITE_JPEG_QUALITY), args.jpeg_quality]
    )
    if not ok:
        continue

    data = encoded.tobytes()
    total_chunks = (len(data) + MAX_CHUNK_PAYLOAD - 1) // MAX_CHUNK_PAYLOA
    if total_chunks == 0:
        continue
```

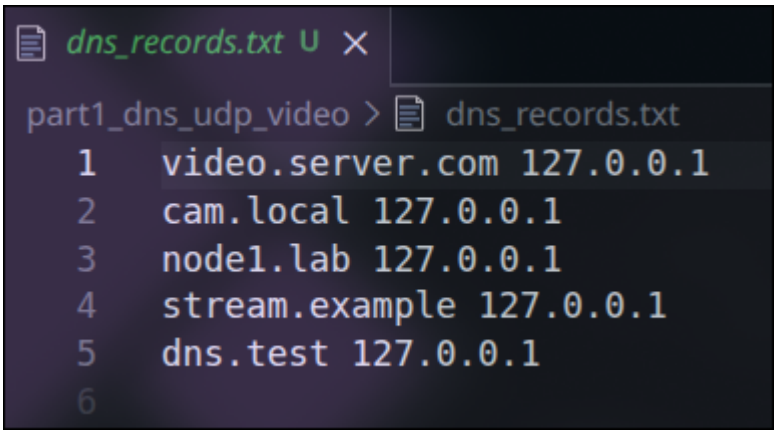
START and KEEPALIVE handling block, frame chunking and datagram send loop - video_server.py

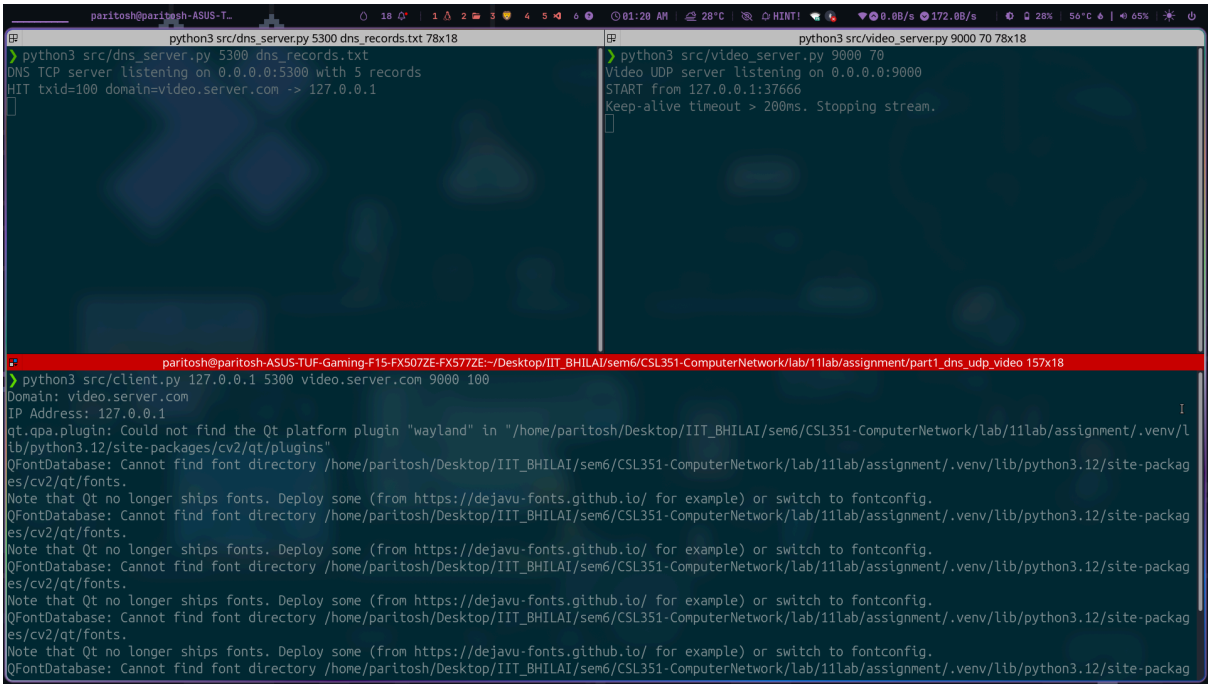
Results

- DNS resolution succeeded and printed domain/IP before streaming
- Live video displayed correctly after **START/KEEPALIVE** sequence
- On client termination, server logs timeout and stops stream
- These results demonstrate both control-plane correctness (DNS + keep-alive) and data-plane continuity (UDP frame stream)

Command line flags:

- dns_server.py -> <port> <records_file>
- video_server.py -> <port> <jpeg_quality>
- client.py -> <dns_ip> <dns_port> <domain> <video_port> <txid>





PART 2 : Design your own network troubleshooting tools

Q1) Create an application to measure RTT.

The RTT measurement tool (echo_client.c & echo_server.c) operates over UDP. The client utilizes the POSIX clock_gettime(**CLOCK_MONOTONIC**, ...) function to capture precise timestamps immediately before sending a datagram and right after successfully receiving the echo.
To handle packet loss, the client socket employs **setsockopt** with **SO_RCVTIMEO** set to a 1-second timeout. If the **recvfrom** call blocks beyond this timeout, the packet is marked as dropped. The client runs in a loop, calculating the difference between the send and receive timestamps in milliseconds for successful probes while tracking the overall packet loss percentage.

- Client sends packets with sequence number and timestamp header
- Server echoes received payload unchanged
- Client computes RTT from send and receive times

RTT_ms = t_recv - t_send / 10^6

Command-line Client arguments:

- server_ip
- port
- num_packets
- packet_size_bytes
- interval_ms
- <server_ip> <port> <num_packets> <packet_size_bytes> <interval_ms>

```c
struct timeval timeout;
timeout.tv_sec = 1;
timeout.tv_usec = 0;
if (setsockopt(sockfd, SOL_SOCKET, SO_RCVTIMEO, &timeout, sizeof(timeout)) < 0) {
    perror("setsockopt");
    close(sockfd);
    return 1;
}
```

receive-timeout setup in echo_client.c

```c
uint64_t send_time_ns = now_ns();
ssize_t sent = sendto(sockfd, send_buf, (size_t)packet_size, 0,
                      (struct sockaddr *)&server_addr, sizeof(server_addr));
if (sent < 0) {
    perror("sendto");
    break;
}

struct sockaddr_in from_addr;
socklen_t from_len = sizeof(from_addr);
ssize_t n = recvfrom(sockfd, recv_buf, (size_t)packet_size, 0,
                     (struct sockaddr *)&from_addr, &from_len);
```

send/recv RTT measurement loop in echo_client.c

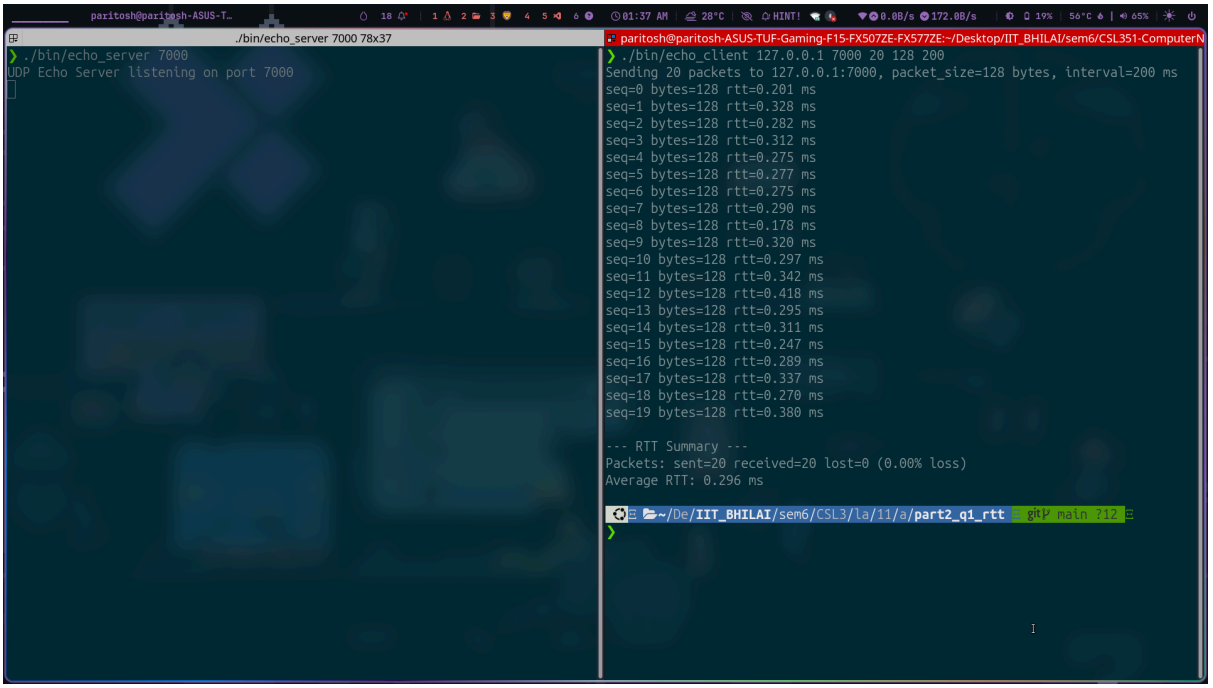
```c
int lost = num_packets - received;
double loss_pct = ((double)lost * 100.0) / (double)num_packets;
double avg_rtt = received > 0 ? total_rtt_ms / (double)received : 0.0;

printf("\n--- RTT Summary ---\n");
printf("Packets: sent=%d received=%d lost=%d (%.2f%% loss)\n", num_packets, received, lost, loss_pct);
printf("Average RTT: %.3f ms\n", avg_rtt);

free(send_buf);
free(recv_buf);
close(sockfd);
return 0;
```

Command line flags:

- echo_server.c -> <port>
- echo_client.c -> <server_ip> <port> <num_packets> <packet_size_bytes> <interval_ms>



Q2) Extend the above application to measure the throughput.

This tool simulates iPerf-like behavior using UDP flooding to measure network capacity and queuing delay over a sustained period. The client (**echo_perf_client.c**) floods the server with 1024-byte payloads at an aggressive rate.
Each packet is embedded with:

1. A sequential packet ID.
2. A high-resolution timestamp of when it was dispatched.

When the server reflects these packets, the client receives them, notes the arrival time, and logs the Round Trip Time (Delay). Every second, the client calculates the instantaneous

Throughput (Total Bytes Received × 8 / 1,000,000) to yield Mbps. The metrics are periodically flushed to a CSV file. A separate Python script (plot_metrics.py) utilizes Matplotlib to parse this CSV and generate time-series graphs for both Throughput (Mbps vs. Time) and Delay (ms vs. Time).

- Packet header includes sequence and send timestamp
- Client transmits continuously for duration_sec
- Replies are used for delay and throughput statistics
- Throughput and delay are logged to two CSV files

Throughput formula:

$$Throughput_{Mbps} = \frac{Bytes_{window} \times 8}{10^6}$$

Delay formula:

$$Delay_{avg,window} = \frac{\sum RTT_i}{N_{window}}$$

```c
typedef struct {
    uint32_t seq;
    uint64_t send_ts_ns;
} PacketHeader;
```

Packet header

```c
char throughput_path[512];
char delay_path[512];
snprintf(throughput_path, sizeof(throughput_path), "%s_throughput.csv", prefix);
snprintf(delay_path, sizeof(delay_path), "%s_delay.csv", prefix);
```

CSV output definition

```c
uint64_t tnow = now_ns();
while (tnow >= next_window_ns) {
    double throughput_mbps = ((double)bytes_received_window * 8.0) / 1000000.0;
    double avg_delay_ms = delay_samples_window > 0 ? delay_sum_ms_window / (double)delay_samples_window : 0.0;

    fprintf(f_thr, "%d,%.6f\n", second_idx, throughput_mbps);
    fprintf(f_dly, "%d,%.6f\n", second_idx, avg_delay_ms);

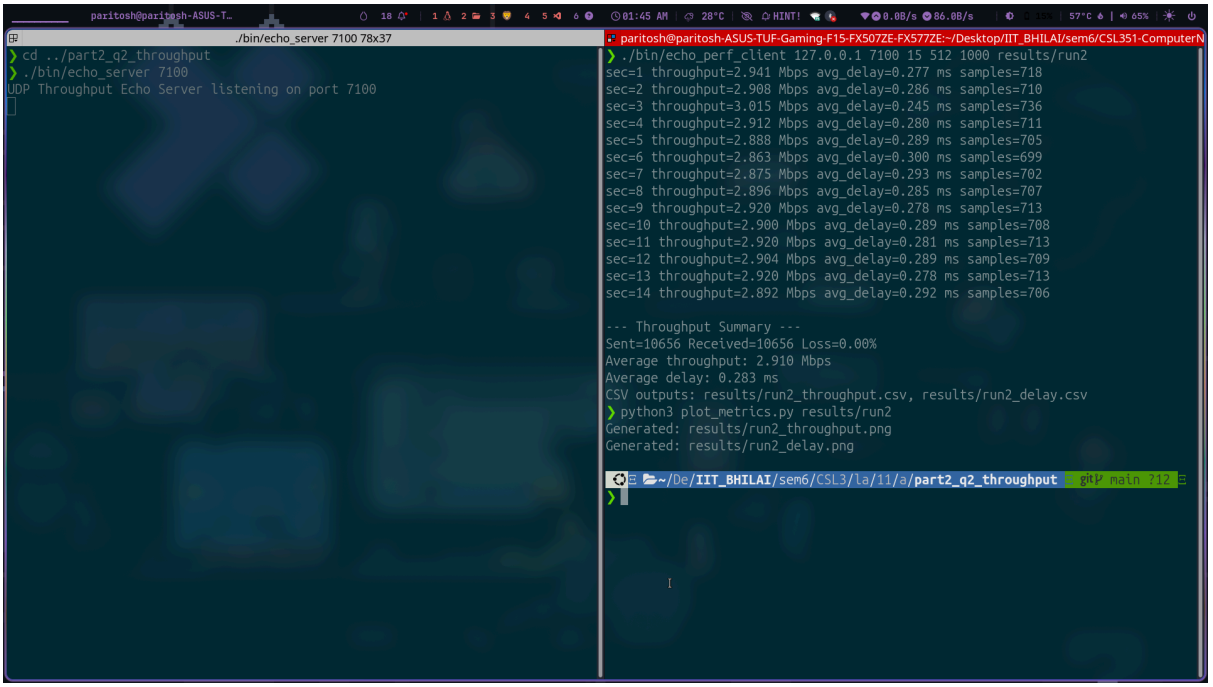
    printf("sec=%d throughput=%.3f Mbps avg_delay=%.3f ms samples=%d\n",
           second_idx, throughput_mbps, avg_delay_ms, delay_samples_window);

    second_idx++;
    bytes_received_window = 0;
    delay_sum_ms_window = 0.0;
    delay_samples_window = 0;
    next_window_ns += 1000000000ULL;
}
```

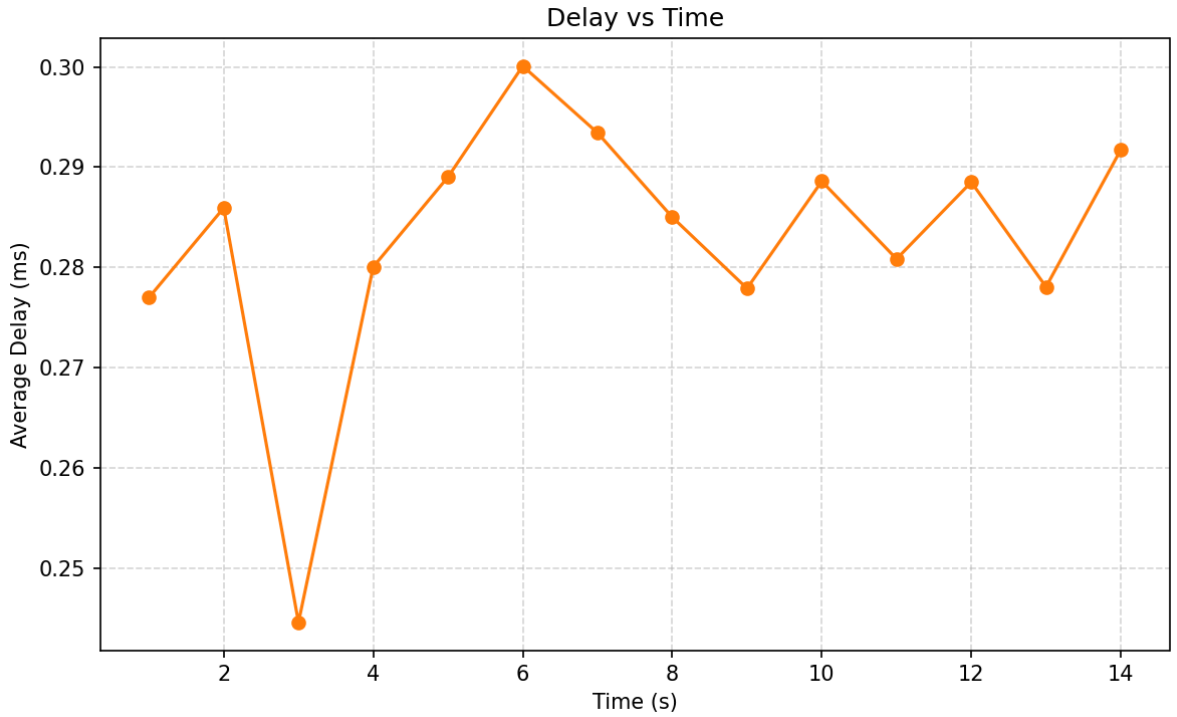
per-second throughput and delay window computation in echo_perf_client.c

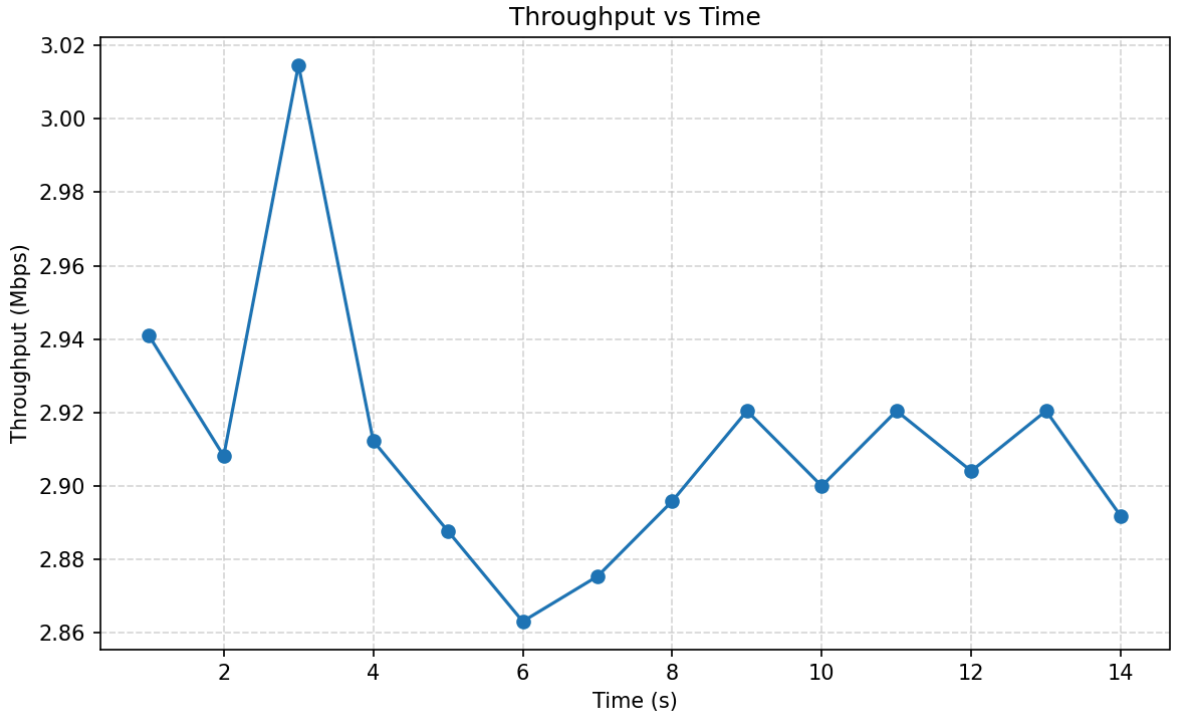
Command line flags:

- Client -> <server_ip> <port> <duration_sec> <packet_size_bytes> <interval_us> <output_prefix>
- Server -> <port>



Below plots are based on the outputs for duration -> 15s, packet size -> 512, interval_use -> 1000





From plot_metric.py

Observations

- With smaller interval_us (higher send rate), throughput rises until bottleneck
- Delay tends to increase when queueing builds
- Packet loss affects measured throughput and average delay stability
- End summary confirms sent/received/loss and average metrics at echo_perf_client.c

Q3) Implement Traceroute

The traceroute utility (**traceroute.c**) is implemented using raw sockets (**SOCK_RAW**) conforming to the **IPPROTO_ICMP** protocol, explicitly avoiding high-level OS utilities.
The core logic relies on manipulating the IP Time-to-Live (TTL) field. The application uses **setsockopt** with the **IP_TTL** flag, starting at a TTL of 1 and incrementing it up to 30. For each TTL, it sends an ICMP Echo Request.
The client then listens for ICMP responses via **recvfrom**. By parsing the raw IP and ICMP headers unpacked from the network buffer, it detects **ICMP_TIME_EXCEEDED** messages (Type 11), identifying the router at that hop. When it finally receives an **ICMP_ECHOREPLY** (Type 0), the program identifies that the final destination has been reached and terminates the trace. It sends 3 probes per hop, calculating the average latency for each router.

Design:

- UDP probe packets are sent with increasing TTL
- ICMP responses are captured from raw socket
- Hop is identified from ICMP sender IP
- Destination is considered reached on ICMP destination unreachable, port unreachable

Timeout handling and output format:

- recv timeout configured via SO_RCVTIMEO in ICMP receive path
- Timeout probes print star symbol
- Three probes per hop are used

```
int raw_sock = socket(AF_INET, SOCK_RAW, IPPROTO_ICMP);
if (raw_sock < 0) {
    perror("socket(raw icmp)");
    fprintf(stderr, "Raw ICMP sockets require elevated privileges. Try: sudo ./traceroute ...\n")
    close(udp_sock);
    return 1;
}
```

raw ICMP socket creation

```c
for (int ttl = 1; ttl <= max_hops && !reached; ++ttl) {
    if (setsockopt(udp_sock, IPPROTO_IP, IP_TTL, &ttl, sizeof(ttl)) < 0) {
        perror("setsockopt(IP_TTL)");
        break;
    }

    printf("%2d  ", ttl);

    struct in_addr hop_addr;
    int have_hop = 0;
    char hop_ip_str[INET_ADDRSTRLEN] = {0};

    for (int probe = 0; probe < DEFAULT_PROBES; ++probe) {
        uint16_t probe_port = (uint16_t)(BASE_DST_PORT + ttl);
        dst_addr.sin_port = htons(probe_port);

        unsigned char payload = (unsigned char)(ttl + probe);
        uint64_t send_ns = now_ns();

        ssize_t sent = sendto(udp_sock, &payload, sizeof(payload), 0,
                              (struct sockaddr *)&dst_addr, sizeof(dst_addr));
        if (sent < 0) {
```

TTL increment and probe loop

```c
static int read_icmp_reply(int raw_sock,
                           uint16_t expected_port,
                           struct in_addr *reply_addr,
                           int *icmp_type,
                           int *icmp_code,
                           int timeout_ms) {
    unsigned char buf[RECV_BUF_SIZE];

    struct timeval tv;
    tv.tv_sec = timeout_ms / 1000;
    tv.tv_usec = (timeout_ms % 1000) * 1000;
    if (setsockopt(raw_sock, SOL_SOCKET, SO_RCVTIMEO, &tv, sizeof(tv)) < 0) {
        perror("setsockopt(SO_RCVTIMEO)");
        return -1;
    }

    while (1) {
        struct sockaddr_in from_addr;
        socklen_t from_len = sizeof(from_addr);
        ssize_t n = recvfrom(raw_sock, buf, sizeof(buf), 0,
                             (struct sockaddr *)&from_addr, &from_len);
        if (n < 0) {
            if (errno == EAGAIN || errno == EWOULDBLOCK) {
                return 0;
            }
            if (errno == EINTR) {
                continue;
            }
            perror("recvfrom(raw_icmp)");
```

ICMP parsing function - traceroute.c

Command line flags:
./bin/traceroute <destination> [max_hops] [timeout_ms]

